

04 — Cross-document consistency

What it detects: documents in the same case that disagree on facts they should share — most importantly, an invoice total that does not match the payment that supposedly settled it.

Code: `services/cross_document_service.py` (`find_cross_document_mismatches`), run by `tasks/document_checks.py` (`run_cross_document_checks`). Persisted to `cross_document_findings` (a **case-level** table — `document_checks` has no case concept).

When it runs

`run_document_checks` (and the failure path of `process_document`) calls `_maybe_enqueue_cross_document_check` , which enqueues `run_cross_document_checks` once **every** document in the case has a terminal `processing_status` (`complete` or `failed`). The check itself:

- 1. loads the case's documents and keeps only those that are `complete` ;
- 2. **deletes any existing findings for the case** and recomputes (this table is *not* append-only, unlike `audit_log`), so a re-run cannot accumulate stale rows;
- 3. if fewer than **2** completed documents exist, writes no findings;
- 4. writes one `cross_document_findings` row per mismatch, audit event `cross_document_check_completed` {`finding_count`} (written even when zero), then requests scoring.

`pipeline_status` waits for a `cross_document_check_completed` event newer than the newest upload before a multi-document case can be scored.

Algorithm

Every unordered pair of completed documents (`itertools.combinations`) is compared on three fields, reading the **normalized** `.value` s (see 02):

Field	Mismatch when	Notes
<code>amount</code>	currencies are both present and differ; or <code>abs(a - b) > max(0.01, 1% of the larger)</code>	Skipped if either side has no amount
<code>date</code>	both parse to a date and the dates differ	ISO dates from extraction; fallback formats exist
<code>issuer</code>	<code>rapidfuzz.fuzz.WRatio</code> (with <code>default_process</code>) < 85	Case/punctuation-insensitive; skipped if either name is empty

Severity depends on the pair's roles

`get_document_role()` classifies each document as `claim` (invoice/quotation/...), `evidence` (`payment_evidence`) or `other` . A claim and its own evidence are **expected** to differ on issuer (vendor vs. bank) and date (invoice date vs. payment date), so:

Field	<code>claim ↔ evidence</code>	any other pair
<code>amount</code>	high (always)	high
<code>date</code> , <code>issuer</code>	low , with an "expected — treat as context" sentence appended	medium

The scoring rules only fire on `medium` / `high` , so an expected claim-vs-evidence date/issuer difference stays visible to a reviewer but adds no risk points.

Output shape

A row in `cross_document_findings` (model `models/cross_document_finding.py`):

```
{
  "case_id": "uuid",
  "field_name": "amount",                // amount | date | issuer
  "finding_type": "cross_document_consistency",
  "severity": "high",                    // low | medium | high
  "description": "Amount differs between 'Invoice.pdf' (9030.00 USD) and 'Evidence.pdf' (7250.00 USD).",
  "document_ids": ["<uuid a>", "<uuid b>"]
}
```

The API returns these as `CrossDocumentFindingSummary` , and derives **one highlight region per involved document** at read time (`field_exception_service.cross_document_regions`), each captioned with what the *other* document said, so the reviewer sees both sides without switching pages.

Known limitations

- Only three fields. Other shared facts (reference numbers, line items, tax IDs) are not compared.
- Pairwise on *all* pairs: a case with three inconsistent documents yields three findings per field.
- A document that failed extraction is excluded, so a case whose only mismatching document failed will not show the mismatch. `pipeline_status` still lets it score.
- Amount tolerance is a flat 1%; there is no per-currency rounding logic and no currency conversion (a currency mismatch is reported, not reconciled).

Risk rules fed

rule_id	Field	Fires on severities	Weight	Severity
<code>cross_doc.amount_mismatch</code>	<code>amount</code>	medium, high	35	high
<code>cross_doc.date_mismatch</code>	<code>date</code>	medium, high	12	medium
<code>cross_doc.issuer_mismatch</code>	<code>issuer</code>	medium, high	15	medium

A case-level rule fires **once per case**, with `count` = number of matching findings.